

Università degli Studi di Firenze
Corso di Laurea Triennale in Informatica

Progettazione e Analisi di un Applicativo di Rete Client/Server TCP/UDP con Misurazione del RTT

Andrea Bertini • Sean Andreini

Progetto per il corso di Reti di Calcolatori

Indice

1	Introduzione	2
2	Uso dell'intelligenza artificiale	2
3	Architettura e Scelte Progettuali	2
3.1	Unificazione dell'Applicativo	2
3.2	Gestione degli Argomenti a Runtime	2
3.3	Portabilità del Codice	3
4	Dettagli Implementativi	3
4.1	Definizione del Pacchetto e Timestamping RFC 8877	3
4.2	La Logica dell'Echo con Modifica del Payload	3
4.3	Gestione dei Timeout e Socket Non Bloccanti	3
5	Test e Analisi	4

1 Introduzione

L'obiettivo di questo assignment è costruire un applicativo client/server che scambia pacchetti usando TCP e UDP.

L'applicazione può funzionare in modalità client (invia richieste) o server (risponde con un echo modificato). Sono state implementate le funzionalità richieste, sia obbligatorie sia opzionali:

- configurazione dei parametri a runtime: porta, protocollo, indirizzo host, numero di pacchetti;
- meccanismo di eco nel server che modifica il payload prima di rispedirlo;
- inclusione nel payload di un timestamp in formato RFC 8877 per calcolare il Round Trip Time;
- opzione per disattivare Nagle su TCP tramite `TCP_NODELAY`.

Il software è stato sviluppato in C e compilato sia su macOS/Linux sia su Windows.

2 Uso dell'intelligenza artificiale

Per alcune parti del codice è stato usato il supporto da parte di Gemini, in particolare per le parti dove non avevamo alcune conoscenze (i punti in cui è stato usato sono segnalati nel codice e nei commenti).

3 Architettura e Scelte Progettuali

3.1 Unificazione dell'Applicativo

Invece di realizzare due programmi separati per TCP e UDP, si è mantenuta un'unica applicazione in `main.c`. La modalità viene scelta a runtime tramite argomenti da riga di comando.

Questa opzione ha permesso di:

1. condividere le strutture dati principali, come il pacchetto e il calcolo del timestamp, riducendo la duplicazione;
2. compilare e gestire un unico sorgente con un `makefile` semplice.

3.2 Gestione degli Argomenti a Runtime

Per leggere i parametri da riga di comando è stata integrata la libreria open-source `argparse`. In questo modo il programma accetta flag come `-p` per la porta, `-u` per UDP, `-l` per server, `-H` per l'indirizzo host, `-n` per il numero di pacchetti e `-N` per disattivare Nagle.

Il programma fornisce anche automaticamente l'aiuto tramite `-help` o `-h`.

3.3 Portabilità del Codice

Per garantire il corretto funzionamento sia su macOS sia su Windows, sono state impiegate delle direttive di precompilazione (`#ifdef _WIN32`). Su sistemi Windows, il programma provvede a caricare dinamicamente le librerie di rete richieste (`Winsock2.h`) all'avvio e a scaricarle alla chiusura del processo tramite la chiamata `WSACleanup()`.

4 Dettagli Implementativi

4.1 Definizione del Pacchetto e Timestamping RFC 8877

Per scambiare messaggi e calcolare l'RTT in modo conforme allo standard, è stata definita la struttura `packet` contenente un payload testuale e una sotto-struttura per il timestamp NTP a 64 bit (diviso in 32 bit per la parte intera dei secondi e 32 bit per la frazione di secondo):

```
1 struct ntp_timestamp {
2     uint32_t seconds;
3     uint32_t fraction;
4 };
5
6 struct packet {
7     struct ntp_timestamp ts;
8     char message[MAX_MESSAGE_SIZE];
9 };
```

Listing 1: Definizione della struttura del pacchetto e dei timestamp conformi a RFC 8877

La conversione del tempo UNIX (calcolato tramite `gettimeofday`) nel formato NTP richiede l'aggiunta di una costante di offset pari a 2208988800UL (la differenza in secondi tra l'epoca NTP del 1900 e quella UNIX del 1970). Per evitare problemi di byte-ordering tra macchine con diversa endianness, i dati numerici del timestamp vengono sistematicamente convertiti in Network Byte Order tramite `htonl()` prima della trasmissione e riconvertiti tramite `ntohl()` alla ricezione.

4.2 La Logica dell'Echo con Modifica del Payload

Per adempiere al requisito di modifica del pacchetto, la logica del server si occupa di estrarre il testo inviato dal client e di sovrascriverlo inserendo il prefisso "[SERVER ECHO]". Ad esempio, se il client invia il messaggio "ciao", il server risponde rispedendo un pacchetto contenente lo stesso timestamp originale ma con messaggio "[SERVER ECHO] ciao".

4.3 Gestione dei Timeout e Socket Non Bloccanti

Nelle comunicazioni UDP, la perdita di un pacchetto porterebbe il client a bloccarsi indefinitamente in attesa di un feedback. Per evitare questa vulnerabilità, è stata configurata l'opzione di socket `SO_RCVTIMEO` con un limite massimo di attesa di 2 secondi sia per la ricezione UDP sia per quella TCP:

```

1 struct timeval timeout = {2, 0}; // 2 secondi di timeout
2 setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

```

Listing 2: Configurazione del timeout di ricezione sul socket

In caso di mancata risposta entro i limiti, la chiamata di lettura si sblocca, permettendo al client di gestire l'errore senza restare congelato.

5 Test e Analisi

I test sono stati eseguiti in locale su 127.0.0.1 usando il programma `bin/main`. Per avere risultati comparabili, il client ha inviato 20 o 100 pacchetti identici con lo stesso messaggio di prova.

- UDP server: `./bin/main -l -u -p 12345`
- UDP client: `printf 'messaggio n' | ./bin/main -H 127.0.0.1 -u -p 12345 -n 20`
- TCP server: `./bin/main -l -p 12345`
- TCP client con Nagle: `printf 'messaggio n' | ./bin/main -H 127.0.0.1 -p 12345 -n 20`
- TCP client senza Nagle: `printf 'messaggio n' | ./bin/main -H 127.0.0.1 -p 12345 -N -n 20`
- UDP senza server: `printf 'messaggio n' | ./bin/main -H 127.0.0.1 -u -p 12346 -n 1`

I risultati misurati sono riportati nella tabella seguente:

Tabella 1: RTT osservati in ms per i test eseguiti su loopback

Modalità	Min	Max	Avg
UDP 20 pacchetti	0.099	0.309	0.122
TCP 20 pacchetti	0.090	0.135	0.108
TCP 20 pacchetti nonagle	0.043	0.206	0.133
UDP 100 pacchetti	0.092	0.274	0.128
TCP 100 pacchetti	0.077	0.162	0.114
TCP 100 pacchetti nonagle	0.082	0.145	0.106

La prova "UDP senza server" ha confermato la vulnerabilità pratica di UDP: il client invia il messaggio e non riceve alcun RTT, perché non c'è nessun server in ascolto pronto a rispondere.

I risultati mostrano che:

- UDP è veloce ma più variabile: sul test da 100 pacchetti l'RTT medio è salito a 0.128 ms e il valore massimo è diventato 0.274 ms.
- TCP con Nagle rimane stabile su 20 e 100 pacchetti, con RTT medi intorno a 0.108–0.114 ms.
- TCP senza Nagle è più variabile su 20 pacchetti (avg 0.133 ms) ma rimane competitivo su 100 pacchetti (avg 0.106 ms), quindi l'effetto varia con la quantità di pacchetti inviati.
- Il test UDP senza server dimostra chiaramente che UDP non offre una garanzia di consegna: se non è presente un server che risponde, l'unico meccanismo è il timeout configurato.